

로드킬 위험구간 예측

공공데이터 기반 위험도 예측 모델과 대화형 GIS 대시보드, LLM 정책 브리핑·RAG 챗봇을 결합한 3개월 캡스톤 프로젝트 구현 현황.

- 대시보드 목업 뼈대 완료
- 베이스라인 파이프라인 구현 완료
- 공공데이터 수집 — 현재 블로커

도로를 잘라 위험도를 매기고, 담당자의 언어로 설명한다

도로망을 500m 구간으로 나눠 예측 단위를 만들고, 위험구간을 지도에서 탐색하며, 담당자가 구간을
고르면 자연어 브리핑과 챗봇 질의응답이 뒤따르는 흐름을 목표로 한다.

① 위험도 예측

도로 구간별 로드킬 위험도를 RandomForest → XGBoost → GNN 순으로 고도화하며 예측한다.

② GIS 대시보드

Streamlit + pydeck으로 위험구간을 지도 위에서 필터링하고 탐색할 수 있게 한다.

③ LLM 브리핑 · 챗봇

선택한 구간의 SHAP 근거를 Claude에 전달해 정책 브리핑을 생성하고, RAG로 질의응답한다.

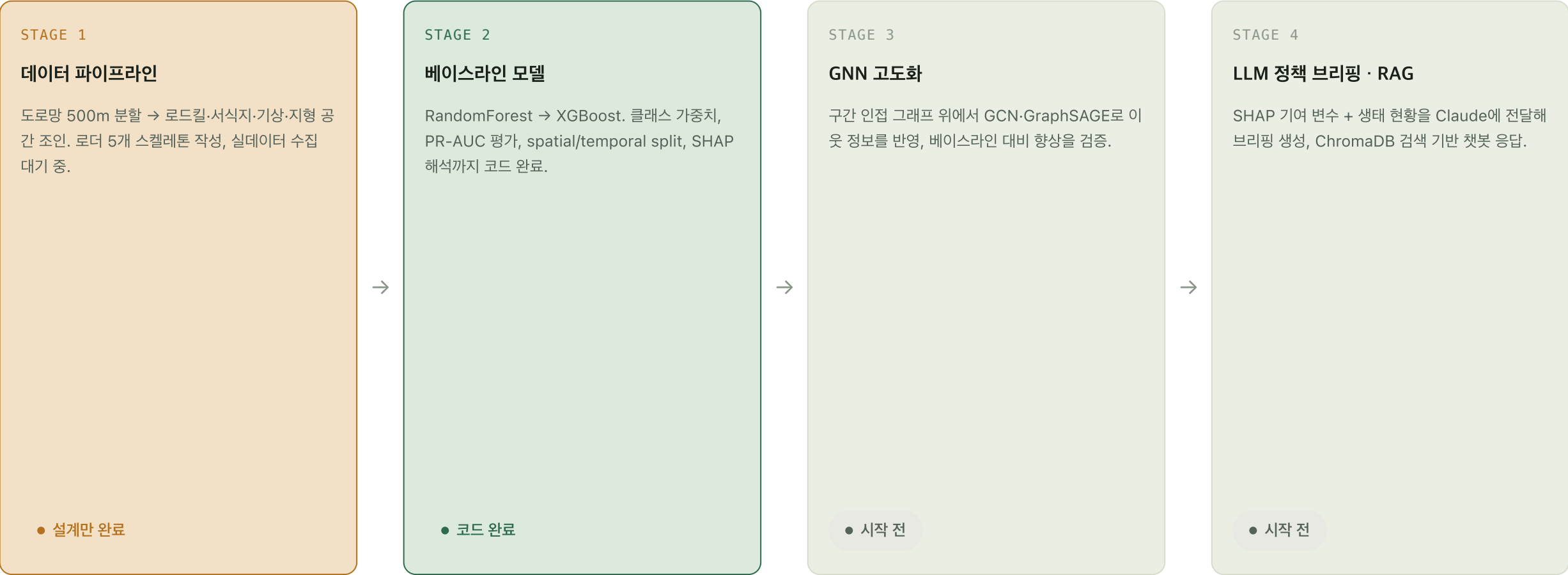
SCOPE - 포함

- 공공데이터 기반 위험도 예측
- 정적 위험지도 + 기상 조건별 위험도
- 정책 브리핑 + RAG 챗봇

SCOPE - 제외 (후속 확장)

4단계 파이프라인

데이터 파이프라인 위에 베이스라인 모델을 올리고, GNN으로 공간적 자기상관을 반영한 뒤, SHAP 근거를 LLM 브리핑·RAG 챗봇이 재사용하는 구조.



5개 공공데이터 출처를 500m 구간에 공간 조인

구간(segment_id)이 예측 단위이자 GNN의 노드가 된다. 아직 원본 데이터는 수집 전이라 로더는 시그니처와 타입힌트만 있고 본문은 NotImplementedError 상태다.

로드킬 발생 지점

TAAS · 국립생태원 에코뱅크

타깃 변수

- 수집 전

서식지 · 생태통로

국립생태원 에코뱅크

생태 피쳐

- 수집 전

도로망 (폭·곡률·AADT)

공공데이터포털 GIS

도로 피쳐

- 수집 전

기상 (강수·안개·기온)

기상청 날씨누리

시간 피쳐

희소 이벤트를 어떻게 학습하고 평가할 것인가

로드킬은 전체 구간 대비 발생 비율이 낮은 희소 이벤트다. 세 가지 결정 기준으로 누수와 낙관 편향을 피했다. 합성 데이터로 전체 흐름(로딩→분할→전처리→학습→평가→SHAP)을 검증했다.

불균형 처리

클래스 가중치

```
class_weight="balanced_subsample" / scale_pos_weight
```

오버샘플링(SMOTE)은 CV 폴드마다 학습 데이터로만 재계산해야 누수를 피할 수 있어 구현이 번거롭다. 가중치 방식은 누수 위험이 없고 단순하다.

평가 지표

PR-AUC 중심

```
average_precision + F1-최적 임계값
```

음성이 압도적으로 많으면 ROC-AUC가 낙관적으로 나온다. 분류 임계값도 고정 0.5 대신 F1을 최대화하는 지점을 탐색한다.

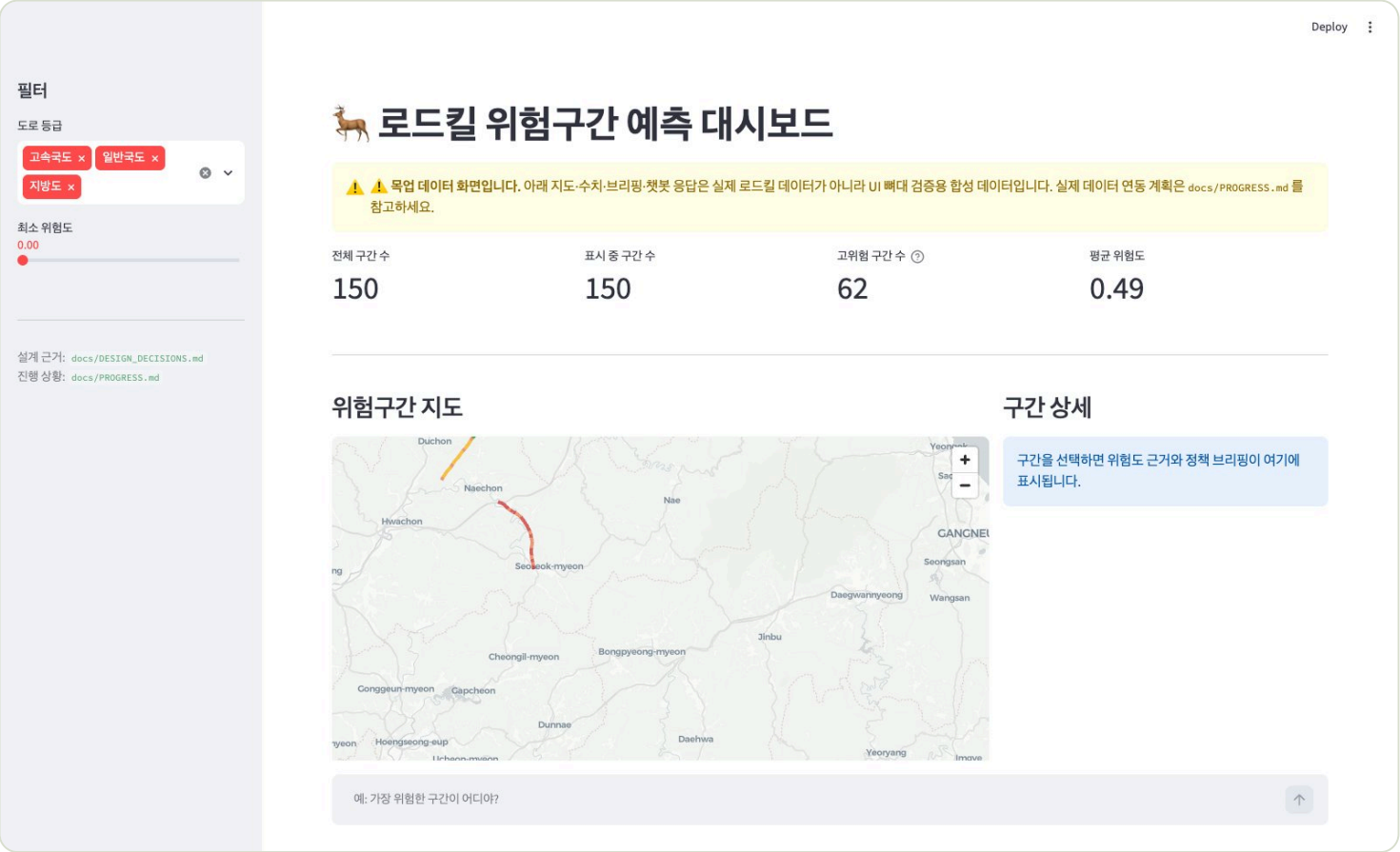
분할 전략

Spatial split 기본값

```
StratifiedGroupKFold(segment_id)
```

목업 데이터로 UI 뼈대부터 먼저 검증

전처리·베이스라인 실데이터 학습을 기다리지 않고, 합성 구간 데이터로 지도·필터·KPI·구간 상세·챗봇 UI를 먼저 만들어 AppTest로 헤드리스 검증까지 마쳤다.



- 01 필터 · KPI
도로 등급 / 최소 위험도 슬라이더, 전체·표시 구간 수, 고위험 구간 수, 평균 위험도.
- 02 위험구간 지도
pydeck 기반, 위험도 4단계(안전·주의·위험·매우위험) 색상 구분, 클릭 선택 지원.
- 03 구간 상세 패널
선택한 구간의 위험도 근거와 정책 브리핑이 표시되는 영역 (LLM 연동 전).
- 04 위험구간 챗봇
현재는 키워드 매칭 기반 목업 응답. RAG·Claude API 연동 예정.

△ 화면의 도로·좌표·위험도·브리핑·챗봇 응답은 전부 합성 데이터입니다. 실제 브라우저에서의 지도 클릭 인터랙션은 아직 직접 확인 전입니다.

전체 9개 모듈 중 3개 완료 · 2개 부분 완료

공공데이터 수집이 전처리·EDA·베이스라인 실학습·GNN 착수로 이어지는 선행 블로커다.

단계	상태	비고
프로젝트 스캐폴딩	● 완료	폴더 구조, requirements.txt, .gitignore
환경 재현성	● 완료	setup.sh / setup.ps1, .python-version
공공데이터 수집	● 시작 전	현재 블로커 — data/raw 비어 있음
전처리 구현	● 설계만 완료	로더 5개 스켈레톤, 21개 함수 NotImplementedError
EDA	● 시작 전	데이터 수집 후 진행
베이스라인 모델	● 코드 완료 / 실학습 전	합성 데이터로 구조 검증 완료
GNN (GCN · GraphSAGE)	● 시작 전	베이스라인 실학습 이후
RAG 파이프라인	● 시작 전	
Streamlit 대시보드	● 목업 뼈대 완료	AppTest 헤드리스 검증 완료, 데이터는 합성

영역별 사용 도구

지형·공간 처리부터 GNN, LLM/RAG, 대시보드까지 전 영역이 requirements.txt에 버전 고정되어 있다.

데이터 처리

- pandas
- geopandas
- shapely
- rasterio

베이스라인

- scikit-learn
- xgboost
- imbalanced-learn
- shap

GNN

- PyTorch
- PyTorch Geometric
- GCN
- GraphSAGE

LLM

- Anthropic Claude API
- claude-haiku-4-5

RAG

- sentence-transformers
- ChromaDB

우선순위 순

1

공공데이터 5종 수집

API 키 발급, 원본 파일을 data/raw/ 에 배치.

• 블로커

2

전처리 구현

실제 데이터 스키마를 보고 로더 5개 채우기, schema.py 컬럼명 재검토.

3

EDA

위험도 분포·결측치·클래스 불균형 정도 확인, 구간-일자 vs 카운트 회귀 결정.

4

베이스라인 실학습

python -m src.models.baseline.train 실행, 실제 PR-AUC 확인.

5

GNN 착수

구간 인접 그래프 구성 후 GCN·GraphSAGE 학습.

6

대시보드 브라우저 검증

지도 클릭 인터랙션을 실제 브라우저에서 육안 확인.

설계 근거와 진행 상황은 저장소 문서에 계속 누적됩니다

새로운 결정을 내리거나 기존 결정을 재검토할 때마다 아래 문서를 갱신합니다.

[README.md](#) — 목표 · 접근 방법 · 설치 및 실행

[docs/PROGRESS.md](#) — 모듈별 상세 현황 · 다음 액션

[docs/DESIGN_DECISIONS.md](#) — 설계 결정의 근거